

ARCHITECTURE AS A LIVING ASSET BASE

How we intend to produce, reuse and defend architecture work

A synthesis note for the architecture community

This note is about how architecture work is done, not about any particular system. It describes a way of working we have designed and partly built: a knowledge base of typed assets, documents generated rather than written, and a system that asks experts what it needs instead of waiting to be told.

July 2026 — for discussion

Contents

1. The problem we are actually solving

Every engagement produces a substantial architecture document. Six months later, almost none of the thinking inside it is retrievable. The document is a PDF on a shared drive; the reasoning that produced it — why this vendor interface rather than direct access, why supervision rather than autonomy, what the effort really turned out to be — is either lost or lives in the head of whoever wrote it. The next engagement starts from a copy of the previous document and inherits its mistakes along with its structure.

Three failures compound. Knowledge decays because a document is a snapshot, not a state. Reuse is copy-paste, so a correction made once is not propagated. And proposals over-claim, because nothing distinguishes a capability we have delivered from one we hope to build — until a technical evaluation committee makes the distinction for us.

The response is not a better document template. It is to stop treating the document as the artefact.

2. The core idea

Assets are the source. Documents are renders. What we hold is a set of small, typed, versioned assets; a proposal or a design dossier is one projection of them at a point in time.

This is the same move we make routinely in engineering and rarely in architecture: separate state from presentation, keep the state structured, and generate the presentation. The consequences are immediate. A correction is made once, in one asset, and every future document inherits it. A claim can be traced to the asset that supports it. And a machine can read the base precisely, because it is reading whole typed objects rather than fragments of prose recovered by similarity.

The base is a single Git repository. Everything in it carries a typed metadata header — identity, status, confidence, applicable phase and domain, owner, date of last review — which is validated automatically. That header is what makes the base machine-consumable; without it, we would have a wiki with better intentions.

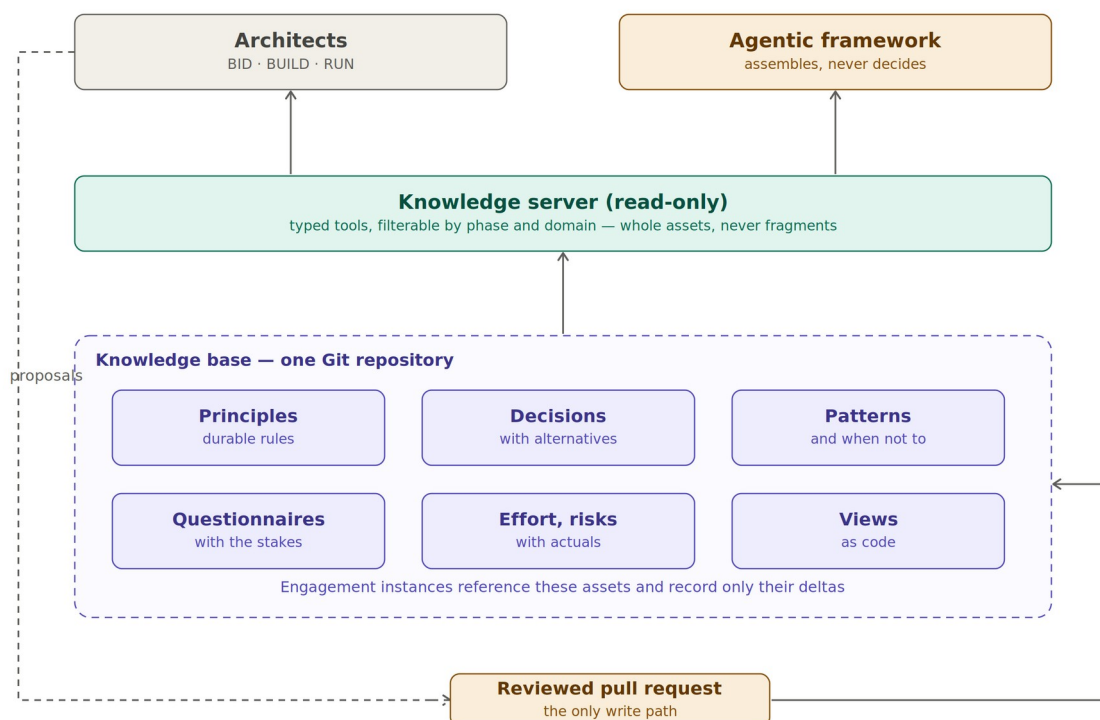


Figure 1 — The base, its read path and its single write path

Agents read the base through a server exposing typed tools, filtered by phase and domain, returning whole assets. They may draft, and they open a reviewed pull request to do so. They never write directly. That constraint is not caution for its own sake: it is what keeps the base a body of reviewed knowledge rather than a corpus that has quietly generated itself.

3. What the base holds

Asset type	What it captures	Why it survives the engagement
Principles	Rules that constrain future choices, each with a clause stating how to verify compliance	They outlive tools and vendors; they are the grid against which any design can be reviewed
Decisions	A choice, the alternatives rejected, and the consequences accepted	Prevents re-litigating settled questions, and re-litigating them badly
Patterns	A repeatable solution, with an explicit section on when not to use it	The 'when not to' is what stops a pattern becoming dogma
Questionnaires	Due-diligence instruments where each question carries its stakes and the cost of a negative answer	Turns a one-off vendor audit into a reusable instrument
Effort models	Ranges with their variance drivers, and actuals recorded per engagement	The drivers matter more than the numbers; without actuals a model is a guess
Risk registers	Risks with mitigations and a record of where each one actually materialised	Lets a bid team argue likelihood from evidence rather than instinct
Views	Diagram generators as code, parameterised	A new engagement changes labels; it does not redraw

Two mechanisms keep the base small enough to be read. Nothing enters the reusable core from a single engagement: an asset is promoted only on its second occurrence, on a different engagement, stripped of anything client-identifying. And nothing is deleted: an asset that no longer holds is marked superseded and linked to its successor, because the trail of a reversed decision is often more instructive than the decision itself.

4. Confidence is data, not tone

Every asset declares whether what it asserts was directly observed, stated by a vendor, or assumed. This is the single most consequential field in the schema, and it exists because of a specific failure mode: a language model will restate an assumption with exactly the same assurance as a fact, and a reader cannot tell the difference. Carrying the epistemic status as data means every tool response, every generated paragraph and every claim in a proposal can be qualified automatically.

Automation enforces it rather than trusting discipline. A vendor-stated assertion that has not been reviewed for a year is downgraded to assumed by the validator. A draft asset that survives two harvests without being promoted is deleted, because a permanently draft asset is noise.

At proposal time this becomes the claims register: one row per statement a customer could hold us to, each carrying an evidence level of proven, designed, vendor-stated or aspirational. Anything aspirational is reworded as a trajectory the architecture supports, or dropped. Moderating a claim becomes a mechanical step in a review rather than an argument between an architect and a bid manager — and it is the cheapest insurance available against failing a technical evaluation for over-claiming.

5. Producing a document, first model: the orchestrated pipeline

The first way of assembling a document suits a single author with a defined scope. Three inputs meet: the reusable base, an engagement brief holding the facts the base cannot know, and a blueprint structure. Four passes follow, with a human decision between each.

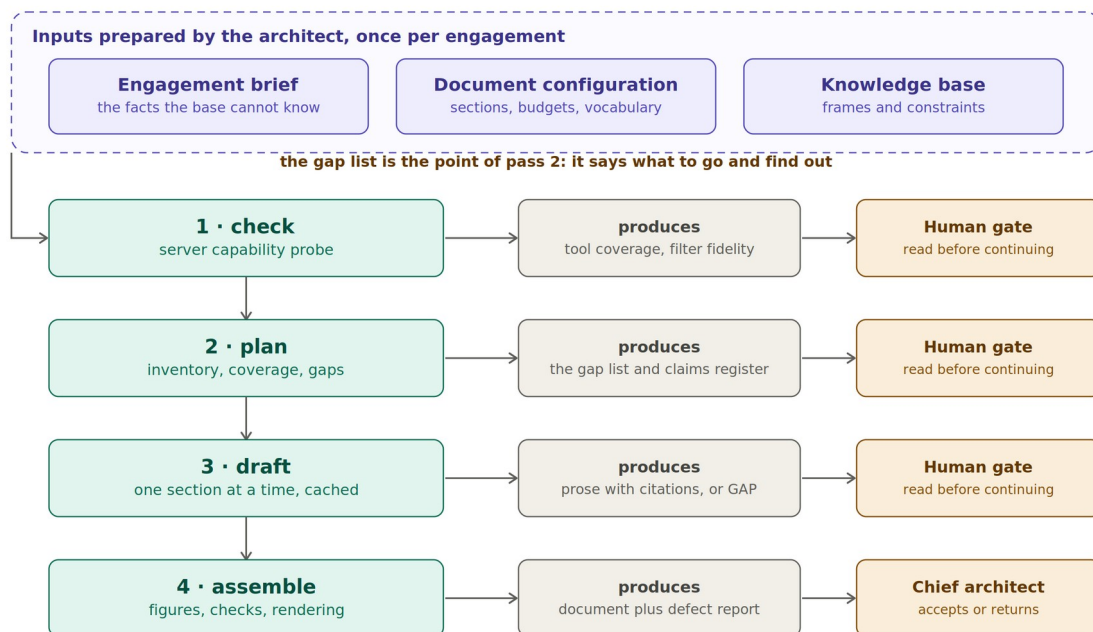


Figure 2 — Four passes, four inputs, and a human gate between each

The pass that matters is the second. It produces an inventory of what the base offers per section, a check of what the brief actually covers, and — the point of the exercise — a list of gaps. A gap is not a defect of the tool; it is the tool telling you what you do not yet know. Each one is either an asset somebody should write, an entry the architect must add to the brief, or a vendor answer that has not arrived. Drafting around a gap with plausible prose is precisely what this apparatus exists to prevent.

Drafting then proceeds section by section, cached, so that iterating on one section does not regenerate the document. Assembly resolves figures by running the generators, runs consistency checks — banned wording, dangling cross-references, undrafted sections — and produces the document with its defect report attached.

The most useful behaviour of this system is not that it writes. It is that it refuses: it reports gaps instead of filling them, returns 'this section cannot be written honestly' instead of improvising, and flags what does not hold together.

6. Producing a document, second model: gap-driven elicitation

The pipeline assumes one author who knows what to say. Real architecture work is collective: several experts hold different pieces, none holds the whole, and they share neither vocabulary nor scope. For that situation the model inverts. The system holds the goal — fill in what is missing for this engagement — and interrogates the experts. It is a chatbot in reverse: the initiative sits with the process, the knowledge with the humans.

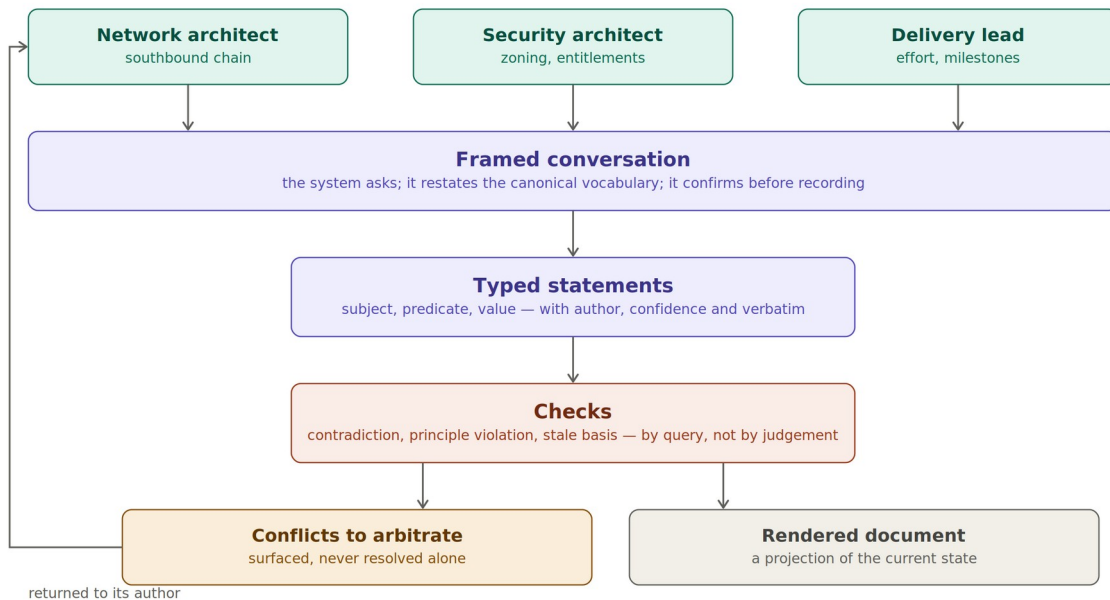


Figure 3 — Multi-expert contribution: framed conversation, typed statements, checks, arbitration

6.1 Statements, not prose

The unit of contribution is not a paragraph. If several experts edit prose in parallel, we collect merge conflicts on text and, worse, semantic contradictions that nobody notices — one section assuming a mechanism another section budgets nothing for. Instead, each contribution is recorded as typed statements: a subject, a predicate from a controlled list, a value, plus the author, the confidence and the expert's own words kept verbatim. Prose is generated from the statements; contradictions become detectable by query.

6.2 The question is where the quality is

A diffuse absence is not a question. Crystallising a gap means producing something answerable in one or two sentences by one identified person, carrying four things: why it matters and what it blocks, the canonical vocabulary to use, the shape of the expected answer, and — where the base has one — what a previous engagement answered.

That last element is the reuse mechanism. Instead of an open question, the expert is shown a default and asked to confirm or diverge. The delta becomes the contribution. It is far cheaper cognitively, and it draws prior engagements into the current one without anyone having to go and read them.

Presenting the canonical vocabulary in the question also solves the hardest problem in this design. If one expert says 'the management cluster' and another 'the administration zone', no contradiction is ever detected because the system does not know they mean the same thing. Serving the term with the question makes the answer attach to the right node by construction — elicitation is the normalisation mechanism.

6.3 What the system checks, and what it must not decide

New statements are checked against the rest: direct contradiction on the same subject and predicate, violation of an applicable principle, reliance on a superseded decision, unsupported claims, terminology drift, scope drift. These are queries over a graph, not judgements of a model.

Where a conflict is found, neither statement is modified or withdrawn. Both remain active, a conflict is raised, and it is routed to the two authors and to the chief architect. The system surfaces; humans arbitrate. This is the same principle we apply to network remediation, turned on our own documents.

7. Where the machine sits, and where it does not

Two separations make this dependable. The first is between what holds state and what reasons. A conversation with an expert unfolds over days: a question goes out on Tuesday, one expert answers Thursday, another is on leave, the chief architect arbitrates the week after. A process that must stay alive through that is broken at the first absence. So the durable process owns the state, the waiting and the audit trail; agent tasks are short, stateless and replaceable; and humans are external actors reached through a mailbox, not functions to be called and awaited.

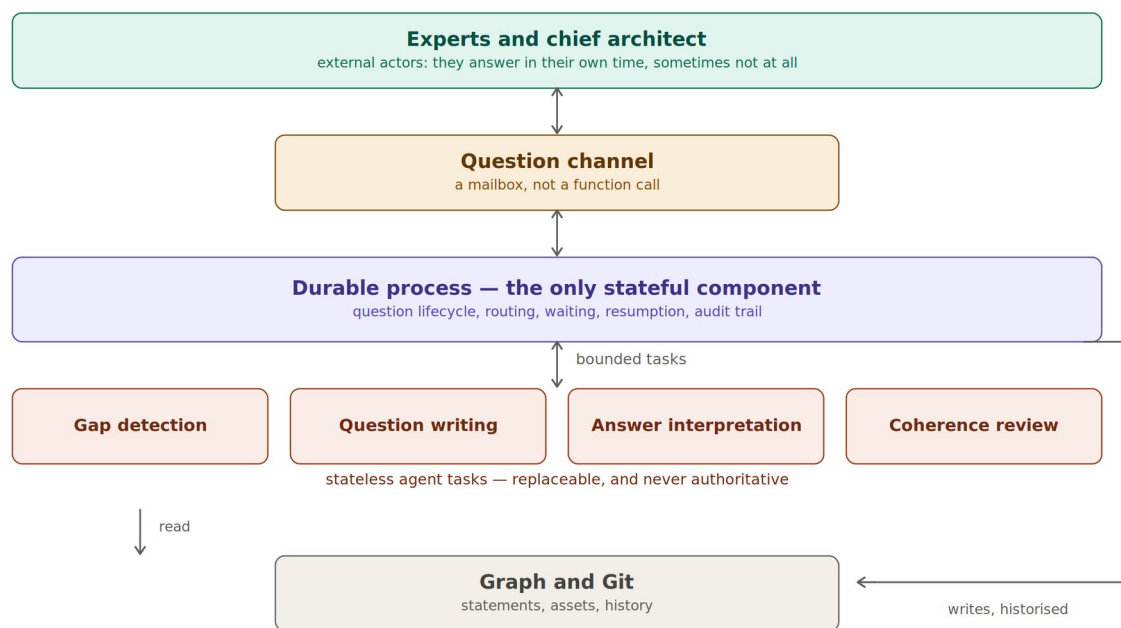


Figure 4 — Durable process, stateless agent tasks, humans as external actors

7.1 The determinism boundary

The second separation decides whether the whole thing is trustworthy.

Concern	How it is done	Why
Detecting a gap	Query	A model asked to find gaps invents some and, worse, misses others silently
Detecting a contradiction	Query	Same; and the result must be reproducible
Phrasing a question	Model	This is writing, which is what it is good

Concern	How it is done	Why
		at
Proposing statements from an answer	Model, then human confirmation	Extraction is a proposal, never a commit
Residual semantic coherence	Model, advisory only	Useful, imperfect, therefore never a gate
Resolving a conflict	Human only	Accountability cannot be delegated to a probabilistic component

One rule follows and admits no exception: a model never writes to the base. It proposes; a human commits. Applied to elicitation this means no statement is recorded without the contributor being shown the extraction and confirming it, with their original words kept alongside so they can say 'that is not what I meant' and prove it.

8. The working principles

Distilled, this is what we are committing to. They are deliberately few.

- **Assets are the source, documents are renders** — what we hold are typed assets; a proposal or a dossier is one projection of them.
- **Confidence is data, not tone** — verified, vendor-stated or assumed travels with every assertion, and automation enforces its decay.
- **Nothing enters the core from one engagement** — promotion requires a second occurrence on a different engagement; the core stays small enough to be read.
- **The system surfaces, humans arbitrate** — conflicts, gaps and inconsistencies are raised, never resolved automatically.
- **The model proposes, a human commits** — extraction, drafting and detection are proposals; commitment is a human act, recorded.
- **Determinism where it matters, generation where it helps** — correctness by query, writing by model. Never the reverse.
- **Ask, do not wait to be told** — the system knows what is missing and goes to find it, rather than waiting for contributions that never come.
- **Harvest, or it is an archive** — effort actuals and materialised risks are the two things no amount of architecture thinking can produce.

9. Roles

Role	What it owns
Contributor	Answers questions in their domain, in their own words. Confirms or corrects the extraction. May decline with a reason, or reroute — a declared unknown is information
Section owner	Completeness and coherence of a section; escalates what cannot be resolved between contributors

Role	What it owns
Chief architect	Arbitrates conflicts, accepts statements into the document, decides deferrals, owns the claims register and therefore the level of promise, signs the harvest
Core owner	Reviews changes to the reusable core in one domain; arbitrates promotions; re-verifies vendor-stated assertions
Maintainer	Repository hygiene, schema, automation, releases

One caution about the chief architect role, because the drift is predictable. They must not answer the questions on the experts' behalf. They know the subject, they are faster, and doing so quietly converts a collective instrument back into a single-author document with extra steps.

10. What this is not, and what is still unproven

It is not an attempt to automate architecture. The system retrieves, structures, drafts, checks and asks. Judgement about what to promise, what to defer and what to refuse stays where it belongs. Nothing here removes the need for an architect; it removes the need for an architect to remember everything and to rewrite what has already been reasoned through.

Several things are honestly unfinished. Subject normalisation across contributors is the hardest problem and is only partly addressed by serving canonical vocabulary with each question. Semantic coherence checking will raise false conflicts and miss real ones, which is why it is advisory. The effort model currently carries no actuals, so it is an educated guess and is marked as such. And the elicitation model is specified and prototyped, not yet in service — the pipeline model is what works today.

The failure mode to watch is not technical. It is a base that fills with drafts nobody promoted, and a harvest ritual that is skipped because the phase ended and everyone moved on. The countermeasure is procedural: the harvest issue is opened at phase kickoff, not at phase end, because an obligation that appears at the end is an obligation that is skipped.

11. How to start

For a colleague who wants to use this on their next engagement, the sequence is short.

- **First** — read the contribution guide; it answers what goes where, and when, in two pages.
- **Second** — fill the engagement brief. It is the input people skip, and a thin brief produces a generic document no amount of prompting will rescue.
- **Third** — run the planning pass and read the gaps before drafting anything. That list is the most valuable output of the whole apparatus.
- **Fourth** — when the phase ends, run the harvest. Record what the effort actually was and which risks actually materialised. Everything else in the base can be reconstructed by thinking harder; those two cannot.

The base, the orchestrator, the knowledge server and the elicitation specification are in the repository. What is missing is use: the assets currently carry the reasoning of one engagement, and they only become a shared body of knowledge on the second.